

NORTHWIND DATA SET USED (with output)

Task 23 – SQL Window Functions Intro

Query 1 - Assign a unique row number to line items ordered by unit price in descending order.

```
mysql> SELECT order_id, product_id, unit_price, ROW_NUMBER() OVER (ORDER BY unit_price DESC) AS row_num FROM mytable LIMIT 5;
```

order_id	product_id	unit_price	row_num
10518	38	263.50	1
10540	38	263.50	2
10541	38	263.50	3
10616	38	263.50	4
10672	38	263.50	5

5 rows in set (0.01 sec)

Query 2 - Rank line items within each order by unit price in descending order using RANK().

```
mysql> SELECT order_id, product_id, unit_price, RANK() OVER (PARTITION BY order_id ORDER BY unit_price DESC) AS price_rank FROM mytable LIMIT 5;
```

order_id	product_id	unit_price	price_rank
10248	72	34.80	1
10248	11	14.00	2
10248	42	9.80	3
10249	51	42.40	1
10249	14	18.60	2

5 rows in set (0.03 sec)

Query 3 - Rank line items within each order by unit price using DENSE_RANK() to avoid skipping ranks on ties.

```
mysql> SELECT order_id, product_id, unit_price, DENSE_RANK() OVER (PARTITION BY order_id ORDER BY unit_price DESC) AS dense_price_rank FROM mytable LIMIT 5;
```

order_id	product_id	unit_price	dense_price_rank
10248	72	34.80	1
10248	11	14.00	2
10248	42	9.80	3
10249	51	42.40	1
10249	14	18.60	2

5 rows in set (0.00 sec)

Query 4 - Compare ROW_NUMBER(), RANK(), and DENSE_RANK() side-by-side ordered by unit price descending.

```
mysql> SELECT order_id, product_id, unit_price, ROW_NUMBER() OVER (ORDER BY unit_price DESC) AS row_num, RANK() OVER (ORDER BY unit_price DESC) AS rnk, DENSE_RANK() OVER (ORDER BY unit_price DESC) AS dense_rnk FROM mytable LIMIT 5;
```

order_id	product_id	unit_price	row_num	rnk	dense_rnk
10518	38	263.50	1	1	1
10540	38	263.50	2	1	1
10541	38	263.50	3	1	1
10616	38	263.50	4	1	1
10672	38	263.50	5	1	1

5 rows in set (0.03 sec)

Query 5 - Find the highest unit price line item for each order using a Common Table Expression (CTE).

```
mysql> WITH RankedItems AS (SELECT order_id, product_id, unit_price, DENSE_RANK() OVER (PARTITION BY order_id ORDER BY unit_price DESC) AS rnk FROM mytable) SELECT order_id, product_id, unit_price, rnk FROM RankedItems WHERE rnk = 1 LIMIT 5;
```

order_id	product_id	unit_price	rnk
10248	72	34.80	1
10249	51	42.40	1
10250	51	42.40	1
10251	22	16.80	1
10251	65	16.80	1

5 rows in set (0.03 sec)

Query 6 - Compare each line item's unit price to the previous line item's unit price within the same order using LAG().

```
mysql> SELECT order_id, product_id, unit_price, LAG(unit_price) OVER (PARTITION BY order_id ORDER BY product_id) AS prev_item_price FROM mytable LIMIT 5;
```

order_id	product_id	unit_price	prev_item_price
10248	11	14.00	NULL
10248	42	9.80	14.00
10248	72	34.80	9.80
10249	14	18.60	NULL
10249	51	42.40	18.60

5 rows in set (0.02 sec)

Query 7 - Calculate the price difference between consecutive line items within the same order using LAG().

```
mysql> SELECT order_id, product_id, unit_price, LAG(unit_price) OVER (PARTITION BY order_id ORDER BY product_id) AS prev_item_price, unit_price - LAG(unit_price) OVER (PARTITION BY order_id ORDER BY product_id) AS price_difference FROM mytable LIMIT 5;
```

order_id	product_id	unit_price	prev_item_price	price_difference
10248	11	14.00	NULL	NULL
10248	42	9.80	14.00	-4.20
10248	72	34.80	9.80	25.00
10249	14	18.60	NULL	NULL
10249	51	42.40	18.60	23.80

5 rows in set (0.04 sec)

Query 8 - Compare each order's total revenue to the previous order's total revenue.

```
mysql> WITH OrderTotals AS (SELECT order_id, SUM(unit_price * quantity * (1 - discount)) AS order_total FROM mytable GROUP BY order_id) SELECT order_id, ROUND(order_total, 2) AS order_total, ROUND(LAG(order_total) OVER (ORDER BY order_id), 2) AS prev_order_total, ROUND(order_total - LAG(order_total) OVER (ORDER BY order_id), 2) AS diff_from_prev_order FROM OrderTotals LIMIT 5;
```

order_id	order_total	prev_order_total	diff_from_prev_order
10248	440.00	NULL	NULL
10249	1863.40	440.00	1423.40
10250	1552.60	1863.40	-310.80
10251	654.06	1552.60	-898.54
10252	3597.90	654.06	2943.84

5 rows in set (0.04 sec)

Query 9 - Retrieve the next line item's unit price within the same order using LEAD().

```
mysql> SELECT order_id, product_id, unit_price, LEAD(unit_price) OVER (PARTITION BY order_id ORDER BY product_id) AS next_item_price FROM mytable LIMIT 5;
```

order_id	product_id	unit_price	next_item_price
10248	11	14.00	9.80
10248	42	9.80	34.80
10248	72	34.80	NULL
10249	14	18.60	42.40
10249	51	42.40	NULL

5 rows in set (0.00 sec)

Query 10 - Calculate the running cumulative total revenue for each line item within an order.

```
mysql> SELECT order_id, product_id, ROUND(unit_price * quantity * (1 - discount), 2) AS line_total, ROUND(SUM(unit_price * quantity * (1 - discount)) OVER (PARTITION BY order_id ORDER BY product_id), 2) AS running_order_total FROM mytable LIMIT 5;
```

order_id	product_id	line_total	running_order_total
10248	11	168.00	168.00
10248	42	98.00	266.00
10248	72	174.00	440.00
10249	14	167.40	167.40
10249	51	1696.00	1863.40

5 rows in set (0.00 sec)

Query 11 - Find the difference between each item's unit price and the average unit price of items in that order.

```
mysql> SELECT order_id, product_id, unit_price, ROUND(AVG(unit_price) OVER (PARTITION BY order_id), 2) AS avg_order_unit_price,
ROUND(unit_price - AVG(unit_price) OVER (PARTITION BY order_id), 2) AS diff_from_avg FROM mytable LIMIT 5;
```

order_id	product_id	unit_price	avg_order_unit_price	diff_from_avg
10248	11	14.00	19.53	-5.53
10248	42	9.80	19.53	-9.73
10248	72	34.80	19.53	15.27
10249	14	18.60	30.50	-11.90
10249	51	42.40	30.50	11.90

5 rows in set (0.03 sec)

Query 12 - Compute a 3-item moving average of unit prices within each order.

```
mysql> SELECT order_id, product_id, unit_price, ROUND(AVG(unit_price) OVER (PARTITION BY order_id ORDER BY product_id ROWS
BETWEEN 2 PRECEDING AND CURRENT ROW), 2) AS moving_avg_price FROM mytable LIMIT 5;
```

order_id	product_id	unit_price	moving_avg_price
10248	11	14.00	14.00
10248	42	9.80	11.90
10248	72	34.80	19.53
10249	14	18.60	18.60
10249	51	42.40	30.50

5 rows in set (0.01 sec)

NOTES

1. Objective & Core Concept

- Goal: Apply ROW_NUMBER, RANK, DENSE_RANK, LAG, and LEAD to solve analytical queries using the Northwind order_details dataset (mytable).
- Core Advantage: Unlike GROUP BY which collapses multiple rows into a single summary, window functions keep all individual row details intact while adding calculated columns alongside them.

2. Ranking Functions Compared

- ROW_NUMBER(): Assigns a strict sequential integer (1, 2, 3, 4) to every row. Never duplicates or skips numbers, even if values tie.
- RANK(): Assigns the same rank to tied values, but skips subsequent rank numbers (1, 2, 2, 4).
- DENSE_RANK(): Assigns the same rank to tied values, but never skips numbers (1, 2, 2, 3).

3. Trend & Value Navigation (LAG vs LEAD)

- LAG(): Reaches back to retrieve data from a preceding row within the same order/partition. Used for calculating line-item price differences or order-over-order revenue growth.
- LEAD(): Looks ahead to retrieve data from a subsequent row. Used for comparing current items against upcoming items.

4. Window Function Statement Structure

Every window query follows this standard pattern:

- Function: What math or tool to apply (ROW_NUMBER(), RANK(), SUM(), LAG()).
- OVER (...): Tells SQL to calculate across a dynamic window of rows instead of collapsing them.
- PARTITION BY: Splits dataset into sub-groups (e.g., grouping calculations by order_id). Resets calculation for each new group.
- ORDER BY: Sets row sequence inside each group (required for ranking, running sums, and trend lookups).

- Frame Limits (ROWS BETWEEN): Defines specific sliding window boundaries (e.g., current row plus 2 preceding rows for a moving average).